

Struts/JSP Monolith → React + Spring Boot Microservices

The Complete Enterprise Migration Guide

Docker • Kubernetes (AKS) • Azure • Kafka • CI/CD • Observability • Security

End-to-End Phases, Team Roles, Migration Patterns, System Design

Real-Time Challenges, Advantages & Disadvantages, Production Issues, Interview Q&A

A practical, simple-language, end-to-end reference for real enterprise modernization projects and interview preparation

Table of Contents

1. Introduction — Why This Migration Guide Matters
2. The Legacy System — Struts/JSP Monolith Architecture (Before)
3. Why Migration Is Needed — Real Business & Technical Drivers
4. The Target Architecture — React + Spring Boot Microservices on AKS/Azure (After)
5. Migration Phases Overview — The Complete End-to-End Roadmap
6. Phase 1: Assessment & Discovery
7. Phase 2: Team Involvement & Roles (Business, Architecture, Development, QA, DBA, DevOps)
8. Migration Pattern Deep Dive — The Strangler Fig Pattern
9. Migration Pattern Deep Dive — Database per Service & Saga Pattern
10. Phase 3: Service Decomposition Strategy
11. Phase 4: Database Migration Strategy
12. Phase 5: API Migration (Struts Actions → REST APIs)
13. Phase 6: Kafka Adoption for Event-Driven Communication
14. Phase 7: Frontend Migration — JSP to React
15. Phase 8: Containerization (Docker)
16. Phase 9: Kubernetes Deployment (AKS)
17. Phase 10: Azure Services Selection
18. Phase 11: CI/CD Setup
19. Phase 12: Observability & Monitoring Setup
20. Phase 13: Security Implementation
21. Phase 14: Production Cutover Strategy
22. Rollback Strategies
23. System Design — Where to Apply Each Pattern (Decision Framework)
24. Real-Time Challenges During Migration
25. Advantages & Disadvantages of the Migration
26. Common Production Issues After Migration & Solutions
27. Real-World Migration Scenarios
28. Interview Questions & Solutions
29. Final Summary & Cheat Sheet

1. Introduction — Why This Migration Guide Matters

1.1 The Scenario This Guide Covers

A large organization has an aging Struts/JSP monolithic web application — built years ago, still running the core business, but increasingly difficult to change, scale, and hire developers for. The organization decides to modernize it to a React frontend, Spring Boot microservices backend, deployed on Docker/Kubernetes (AKS) in Azure. This exact scenario is one of the most common real-world enterprise projects a Java developer will encounter mid-career.

1.2 Why This Is Genuinely Hard (Not Just "Rewrite the Code")

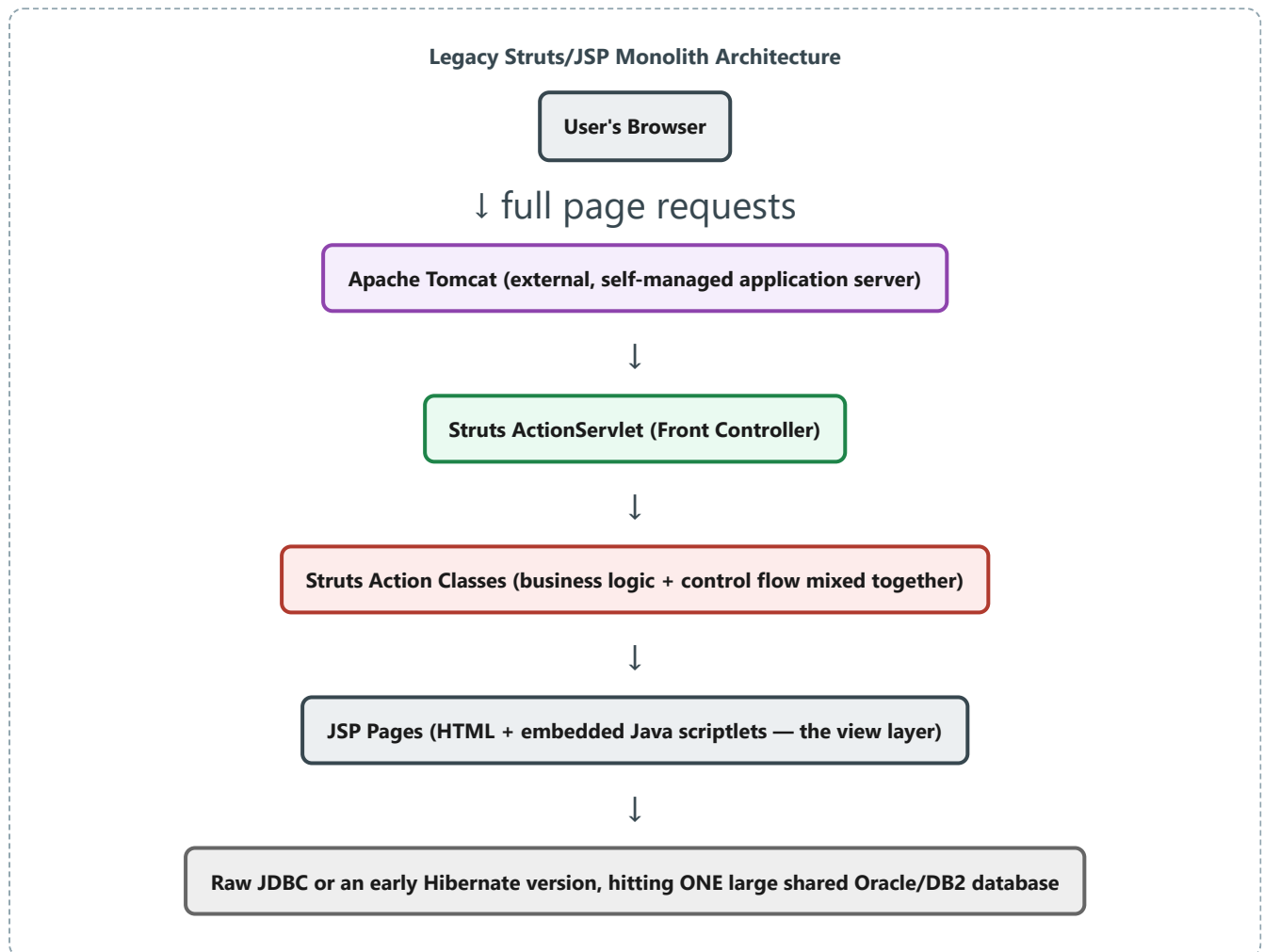
A real migration like this isn't a coding exercise — it's a multi-month (often multi-year) organizational program involving business stakeholders, architects, multiple development teams, QA, DBAs, and DevOps, all coordinating around a system that must KEEP RUNNING and serving real customers throughout the entire process. Understanding this full complexity — not just the target technology — is what this guide covers.

1.3 How This Guide Is Structured

- **Sections 2-4:** Understand the starting point (legacy architecture), the real reasons to migrate, and the target end-state architecture.
- **Sections 5-22:** Walk through the complete end-to-end migration roadmap, phase by phase, including the core migration patterns (Strangler Fig, Database per Service, Saga) woven in at the right points.
- **Section 23:** A system design decision framework — when to use which pattern.
- **Sections 24-27:** Real-time challenges, honest advantages/disadvantages, production issues, and real-world scenarios.
- **Sections 28-29:** Interview questions with full solutions, and a final cheat sheet.

2. The Legacy System — Struts/JSP Monolith Architecture (Before)

2.1 What a Typical Struts/JSP Application Looks Like



2.2 Real Characteristics of This Legacy System

Characteristic	Real Impact
One massive WAR file deployment	Every single change, no matter how small, requires rebuilding and redeploying the ENTIRE application
Business logic scattered across Action classes and JSP scriptlets	No clean separation of concerns (recap: Spring & Spring Boot guide, Section 2) — hard to test, hard to reason about
One shared database for everything	Tables from unrelated business areas are tightly coupled — a schema change anywhere risks breaking something elsewhere
Manual or semi-automated deployment	Deployments are risky, infrequent (often monthly or quarterly), and require significant manual verification

Tightly coupled to a specific application server version	Upgrading Java/Tomcat versions is a large, risky undertaking, often deferred for years
No automated test coverage (common in older codebases)	Every change carries real regression risk, discovered only through manual QA

2.3 A Real Struts Action Class (What Developers Actually Maintain Today)

```
public class TransferFundsAction extends Action {
    public ActionForward execute(ActionMapping mapping, ActionForm form,
                                HttpServletRequest request, HttpServletResponse response) {
        TransferForm transferForm = (TransferForm) form;
        // Business logic, validation, and control flow ALL mixed together here
        Connection conn = DriverManager.getConnection(DB_URL, USER, PASS); // raw JDBC, manual connection
management
        // ... 200+ lines of mixed logic ...
        request.setAttribute("result", result);
        return mapping.findForward("success"); // manually wired navigation, defined in struts-config.xml
    }
}
```

This exact class is precisely the kind of code a modernization project needs to decompose — the business logic buried inside needs to be extracted into a clean Spring Boot Service layer, and the JDBC code replaced with Spring Data JPA (JDBC/Hibernate/JPA guide, full detail).

3. Why Migration Is Needed — Real Business & Technical Drivers

3.1 Business Drivers

Driver	Real Impact
Slow time-to-market	New features take months due to the monolith's size/risk — competitors with modern architectures ship faster
Rising infrastructure cost	Aging on-premises hardware needs expensive refresh cycles; cloud offers better cost efficiency at scale
Talent/hiring difficulty	Struts is a legacy framework — hiring and retaining developers skilled in it is increasingly hard and expensive
Scalability limits	The monolith must be scaled as ONE unit, even if only one small part actually needs more capacity
Customer expectations	Users expect modern, responsive UIs (React-style SPAs) — JSP-rendered pages feel dated

3.2 Technical Drivers

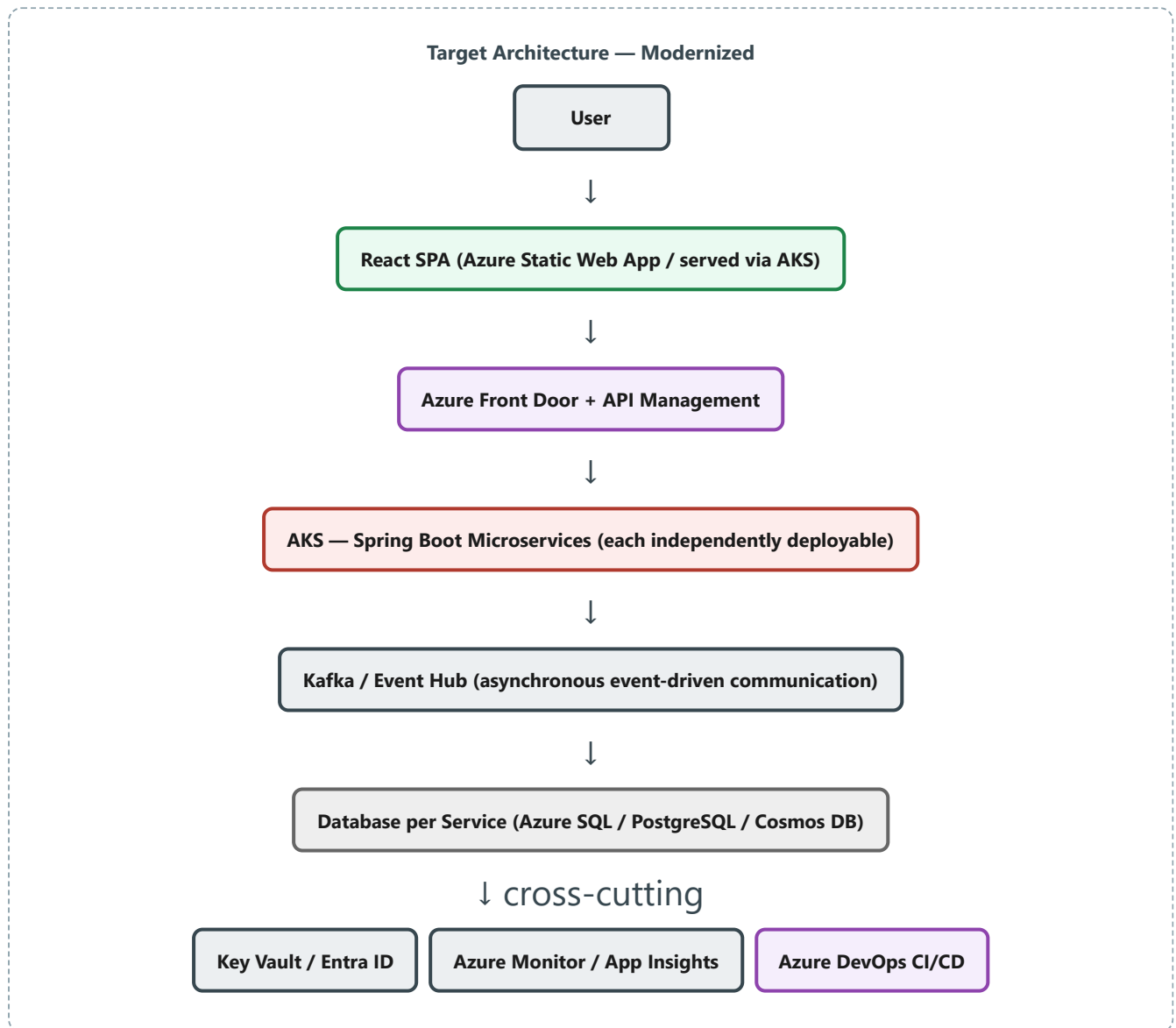
Driver	Real Impact
Vendor/framework end-of-life risk	Struts has had serious historical security vulnerabilities (e.g., CVE-2017-5638) — an unmaintained framework is a real security liability
Deployment risk and downtime	Deploying the entire monolith for one small fix risks breaking unrelated functionality
No independent team ownership	Multiple teams working in the same codebase causes merge conflicts and coordination overhead
Poor observability	Legacy systems often lack modern distributed tracing/metrics, making production issues hard to diagnose

3.3 The Honest Counter-Question — Is Migration Always the Right Call?

A responsible architect must also ask: is a full migration genuinely justified, or would targeted modernization (e.g., just upgrading Struts version, adding better monitoring) solve the immediate pain more cheaply? Full microservices migrations are expensive and risky — Section 25 (Advantages & Disadvantages) covers this honestly. This guide assumes the organization has already made the considered decision to migrate, and focuses on doing it well.

4. The Target Architecture — React + Spring Boot Microservices on AKS/Azure (After)

4.1 The Full Target State Architecture



This is EXACTLY the same full cloud-native architecture already covered in complete depth in the AWS/Azure Cloud Architecture guide and the Traditional/Hybrid/Full Cloud guide — this migration guide's unique focus is the JOURNEY from the legacy state (Section 2) to this target state, not the target architecture in isolation.

4.2 What Changes, Concretely, for Each Layer

Layer	Before (Legacy)	After (Target)
Frontend	JSP pages, full page reloads	React SPA, API-driven, no full page reloads (Section 14)
Backend	One Struts monolith	Multiple independent Spring Boot microservices (Section 10)

Data Access	Raw JDBC / early Hibernate	Spring Data JPA, database per service (Section 11)
Communication	In-process method calls	REST APIs + Kafka events (Sections 12-13)
Deployment	Manual WAR deployment to Tomcat	Docker containers on AKS, automated CI/CD (Sections 15-18)
Infrastructure	On-premises servers	Azure managed services (Section 17)
Observability	Basic log files	Full distributed tracing/metrics/dashboards (Section 19)

5. Migration Phases Overview — The Complete End-to-End

Roadmap

5.1 The Full Real Migration Roadmap



11. Decommission Legacy System

5.2 Real Timeline Expectations

A migration of this scale for a genuinely large enterprise monolith typically takes **12-36 months**, executed incrementally (never a "big bang" — Section 21's rollback strategies explain why), with the legacy and new systems running in parallel for much of that period. Organizations that expect this to take "a few sprints" are setting themselves up for a failed, rushed migration.

6. Phase 1: Assessment & Discovery

6.1 What Happens in This Phase

Before writing a single line of new code, the organization must deeply understand what it's actually migrating — the monolith's real business capabilities, hidden dependencies, data flows, and technical debt. Rushing this phase is one of the single biggest causes of real migration project failure.

6.2 Real Assessment Activities

- **Codebase analysis:** map every Struts Action to the business capability it implements — often using static analysis tools to find hidden dependencies between modules.
- **Database analysis:** identify which tables belong to which business domain, and where cross-domain foreign keys/joins create tight coupling that will need to be untangled (Section 11).
- **Traffic/usage analysis:** which features are actually used heavily (migrate/optimize first) versus rarely used (candidates for simple rehosting or even retirement).
- **Dependency mapping:** external systems, batch jobs, and integrations the monolith touches that must keep working throughout the migration.
- **Risk assessment:** which business capabilities are the highest-risk to migrate (core revenue-generating flows) versus lowest-risk (internal admin tools).

6.3 Real Output of This Phase

The Assessment phase should produce a concrete **Domain Map** — a documented breakdown of the monolith into candidate business capabilities/bounded contexts (exactly the Domain-Driven Design exercise from the Microservices guide, Section 9.3) — which becomes the direct input to Service Decomposition (Section 10) and the migration wave planning (Section 20's sequencing).

7. Phase 2: Team Involvement & Roles (Business, Architecture, Development, QA, DBA, DevOps)

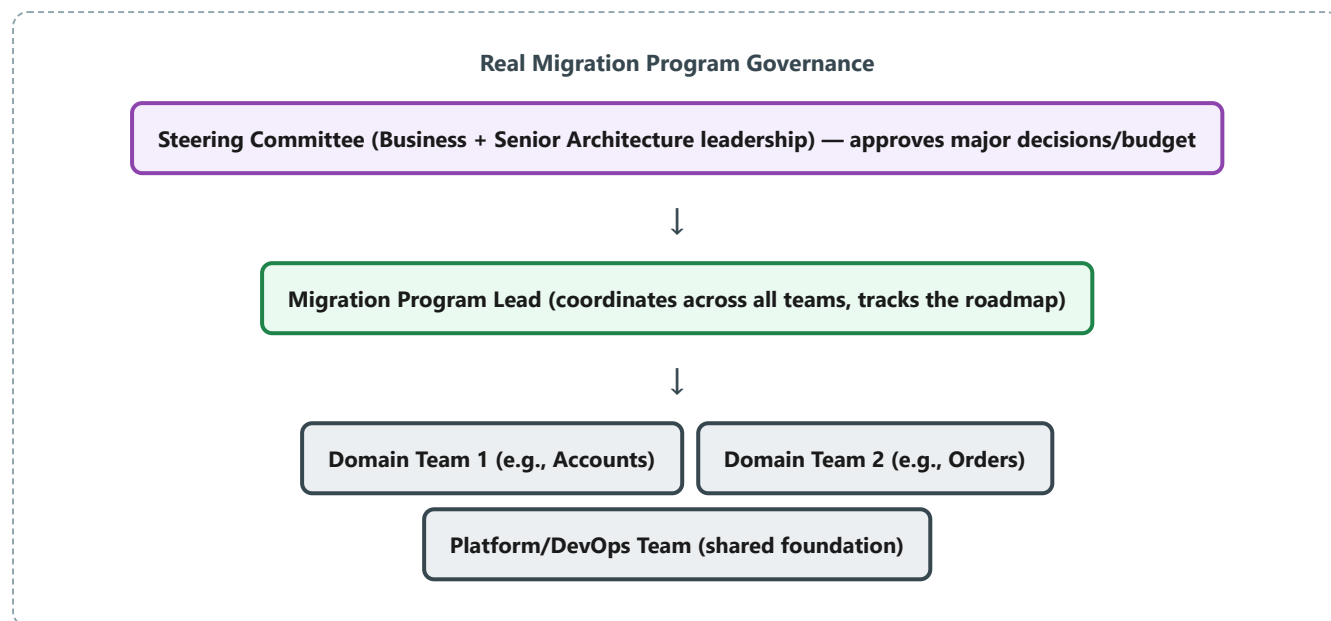
7.1 Why This Is Genuinely a Cross-Functional Program, Not Just a Dev Task

A migration of this scale touches every part of the organization — the mistake many failed migrations make is treating it as a purely engineering initiative without the necessary business and operational partnership.

7.2 Real Team Roles and Responsibilities

Team	Real Responsibilities in This Migration
Business Stakeholders	Prioritize which business capabilities/modules migrate first based on business value/risk; approve any temporary feature freezes during risky cutover windows; own the final go/no-go decision for cutover (Section 20)
Architecture Team	Owns the target architecture design (Section 4), service decomposition boundaries (Section 10), and technology standards applied consistently across all new microservices
Development Teams	Implement the new Spring Boot microservices and React frontend, working domain-by-domain per the migration wave plan
QA Team	Builds automated regression test suites (often the FIRST real investment, since legacy systems often lack them) to safely validate that migrated functionality behaves identically to the legacy system
DBA Team	Plans and executes the database migration/splitting strategy (Section 11), including data synchronization during the parallel-running period
DevOps/Platform Team	Builds the CI/CD pipelines, Kubernetes infrastructure, and observability platform that every migrated service will use (Sections 15-19)
Security Team	Reviews the new architecture's security posture, ensures compliance requirements carry over correctly (Section 19)

7.3 Real Governance Structure



Real scenario: "Our migration program ran a weekly cross-functional sync involving one representative from each team (Business, Architecture, Dev, QA, DBA, DevOps) — this single recurring meeting caught more real cross-team blockers early than any amount of documentation ever could, because database schema changes, new API contracts, and QA test coverage gaps all surfaced and got resolved in the SAME room, not discovered weeks later."

8. Migration Pattern Deep Dive — The Strangler Fig Pattern

8.1 Recap and Why It's THE Central Pattern for This Entire Migration

Already introduced conceptually in the Microservices guide (Section 9) — the Strangler Fig Pattern incrementally routes traffic for specific features away from the legacy monolith to new microservices, one capability at a time, while the legacy system keeps running for everything not yet migrated. This is THE foundational pattern underlying this entire guide's migration approach, because it avoids the catastrophic risk of a "big bang" cutover.

8.2 Applying Strangler Fig to This Specific Struts/JSP Migration

Strangler Fig Applied — Step by Step for This Migration

Step 0: All traffic → Struts Monolith

↓ introduce a routing layer (reverse proxy / API Gateway) in front of everything

Routing Layer introduced — still 100% to Monolith

↓ Step 1: migrate ONE low-risk capability (e.g., "Customer Profile") to a new Spring Boot service

Monolith (everything else)

NEW Customer Profile Microservice

↓ routing layer sends /profile/* to the new service, everything else still to the monolith

Repeat for each subsequent capability, in order of risk (lowest first)

↓ eventually

Final State: Monolith fully "strangled" — decommissioned entirely

8.3 Real Routing Layer Implementation

```
# Real Azure API Management routing rule during migration — path-based cutover
Path /profile/* → NEW Customer Profile Microservice (AKS)
Path /orders/* → NEW Order Microservice (AKS) # migrated in a later wave
Path /* → LEGACY Struts Monolith (still running, unmigrated features)
```

This single routing configuration change is literally how a capability "moves" from legacy to modern — no risky data cutover event, no customer-facing downtime, just a routing rule update that can be instantly reverted if something goes wrong (Section 21's rollback strategy).

8.4 Why This Specific Order Matters — Migration Wave Sequencing

Wave	What Migrates	Why This Order
Wave 1 (Pilot)	A small, low-risk, low-traffic module (e.g., a static content/FAQ page)	Validates the entire pipeline (CI/CD, AKS, monitoring) end to end with minimal business risk
Wave 2	Read-heavy, less critical features (e.g., "View Profile," reporting screens)	Builds real confidence handling genuine production traffic, still relatively low risk if something goes wrong
Wave 3+	Core, high-value, high-traffic business capabilities (e.g., checkout, fund transfer)	Tackled only once the team has multiple successful migrations' worth of real, proven experience

9. Migration Pattern Deep Dive — Database per Service & Saga Pattern

9.1 The Core Migration Challenge These Patterns Solve

The legacy monolith has ONE shared database where a "Customer Profile" update and an "Order" creation could easily happen in a single local ACID transaction. Once split into separate microservices with separate databases (Section 11), that single transaction no longer exists — Database per Service and Saga together are how real migrations solve this without sacrificing data correctness.

9.2 Database per Service — Applied to This Migration

Splitting the Monolith's Shared Database

BEFORE: One Shared Oracle/DB2 Database
(customer, order, payment, inventory tables, all joined freely)

↓ split by bounded context (Section 6.3's Domain Map)

Customer DB

Order DB

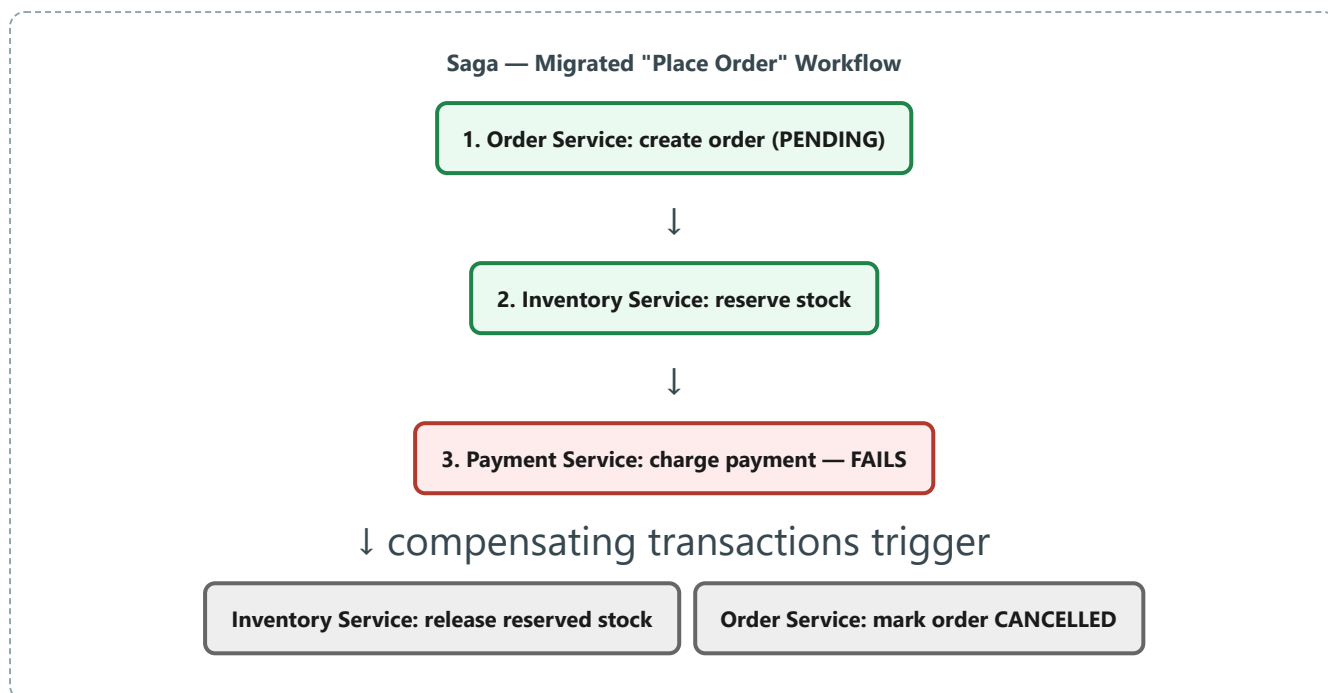
Payment DB

Inventory DB

Full detail on WHY this pattern matters (independent deployability, no cross-service coupling through shared tables) is already covered in the Microservices guide (Section 27) — this section focuses on the MIGRATION-SPECIFIC challenge: how do you actually SPLIT an existing, populated, live production database without data loss or extended downtime (Section 11 covers the real strategies for this).

9.3 Saga Pattern — Applied to a Real Migrated Workflow

Consider "Place Order" — in the legacy monolith, this was one local transaction: create the order row, decrement inventory, and record the payment, all in one commit. After migration, these are THREE separate microservices with separate databases — the Saga pattern (Microservices guide, Section 22) replaces that single transaction with a sequence of local transactions plus compensating actions.



9.4 The Real Migration-Specific Risk — Dual Writes During the Transition

During the parallel-running period (Strangler Fig, Section 8), some capabilities may still write to the legacy shared database WHILE new microservices write to their own separate databases — this dual-write period is a genuinely risky time requiring careful synchronization (often via Change Data Capture, CDC) to keep both representations of the data consistent until the legacy path is fully retired for that capability.

10. Phase 3: Service Decomposition Strategy

10.1 How Real Teams Decide Service Boundaries

Using the Domain Map from the Assessment phase (Section 6.3), the Architecture team applies Domain-Driven Design principles (Microservices guide, Section 9.3) to identify natural "bounded contexts" — business capabilities that have their own data, their own business rules, and change for their own independent reasons.

10.2 Real Decomposition Example for This Migration

Legacy Struts Module	New Microservice(s)	Why This Boundary
CustomerProfileAction, CustomerSearchAction	Customer Service	Owns customer identity/profile data exclusively
OrderCreateAction, OrderHistoryAction, OrderCancelAction	Order Service	Owns the order lifecycle end to end
PaymentProcessAction, RefundAction	Payment Service	Isolated for PCI-DSS compliance scope reduction (fewer services touching card data)
InventoryCheckAction, StockUpdateAction	Inventory Service	High-frequency, independent scaling needs during sales events
ReportGenerationAction (rarely used, complex)	Kept in a "Legacy Reporting" service, migrated LAST or retired	Low business value relative to migration effort/risk — a real, honest prioritization call

10.3 Common Real Decomposition Mistakes

- **Decomposing by technical layer instead of business capability** (e.g., a "Validation Service" or a "Database Access Service") — this creates chatty, tightly-coupled services rather than genuinely independent ones.
- **Making services too fine-grained too early** ("nanoservices") — creating excessive operational overhead for boundaries that don't yet have proven independent scaling/team-ownership needs.
- **Ignoring existing team structure** (Conway's Law, Microservices guide Section 40.2) — service boundaries that don't align with how teams are actually organized cause ongoing coordination friction.

11. Phase 4: Database Migration Strategy

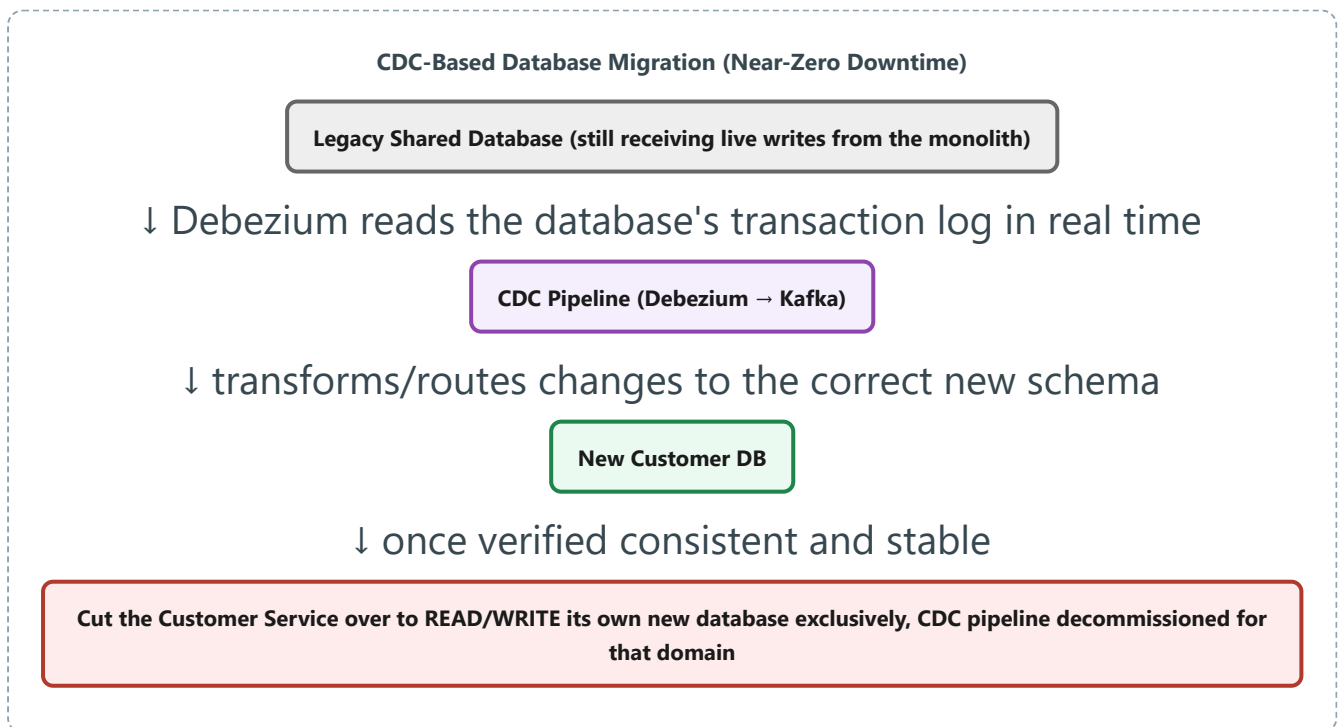
11.1 The Central Challenge — Splitting a Live, Populated Database

Unlike a brand-new microservices project starting with empty databases, THIS migration must split years of real, live production data out of one shared Oracle/DB2 database into multiple independent databases, without data loss and with minimal downtime.

11.2 Real Database Migration Approaches

Approach	How It Works	Real Trade-Off
Big Bang Database Split	Take a full downtime window, migrate all data at once to the new split databases	Simpler to reason about, but requires real downtime and is high-risk if something goes wrong mid-migration
Change Data Capture (CDC)-Based Sync	Use a CDC tool (e.g., Debezium) to continuously replicate changes from the legacy database into the new per-service databases in near-real-time, running in parallel	Enables zero/minimal downtime, but adds real complexity in schema transformation logic and consistency verification
Dual-Write (Application-Level)	The application itself writes to both the old and new databases during the transition	Simpler infrastructure-wise, but risks the two databases drifting out of sync if not implemented very carefully (Section 9.4)

11.3 Real CDC-Based Migration Flow



11.4 Real Data Validation Before Cutover

Before trusting a newly split database, real teams run automated reconciliation scripts comparing row counts, checksums, and sample record-level comparisons between the legacy source and the new target database — any discrepancy is investigated and resolved BEFORE that service is allowed to become the source of truth.

12. Phase 5: API Migration (Struts Actions → REST APIs)

12.1 The Real Transformation Pattern

Every Struts Action class (Section 2.3) must be decomposed into: a clean REST Controller (HTTP concerns), a Service layer (extracted business logic, now properly `@Transactional`), and a Repository (Spring Data JPA replacing raw JDBC) — exactly the layered architecture from the Spring & Spring Boot guide, Section 11.

12.2 Real Before/After Example

```
// BEFORE – legacy Struts Action (Section 2.3), everything mixed together
public class TransferFundsAction extends Action {
    public ActionForward execute(...) {
        // 200+ lines: validation, JDBC, business rules, navigation, all mixed
    }
}

// AFTER – properly decomposed Spring Boot layers
@RestController
@RequestMapping("/api/v1/accounts")
public class TransferController {
    private final TransferService transferService;

    @PostMapping("/transfer")
    public ResponseEntity<TransferResponse> transfer(@Valid @RequestBody TransferRequest request) {
        return ResponseEntity.ok(transferService.transfer(request)); // thin controller, HTTP concerns only
    }
}

@Service
@RequiredArgsConstructor
public class TransferService {
    private final AccountRepository accountRepository;

    @Transactional
    public TransferResponse transfer(TransferRequest request) {
        // clean, testable business logic, Spring Data JPA instead of raw JDBC
    }
}
```

12.3 Real API Contract Compatibility During Migration

Since the React frontend (Section 14) and the new API must work TOGETHER, and often the legacy JSP frontend still needs to keep working for not-yet-migrated features, real teams often design the new REST APIs with careful attention to API versioning (API Development guide, Section 10) so both old and new frontends can coexist safely during the transition.

12.4 Real Contract Testing

Before cutting a Struts Action over to its new REST API equivalent, real teams write Consumer-Driven Contract tests (API Development guide, Section 39.3) comparing the new API's actual response shape against what the frontend genuinely

expects — catching subtle behavioral differences (e.g., a slightly different error message format) before they reach real users.

13. Phase 6: Kafka Adoption for Event-Driven Communication

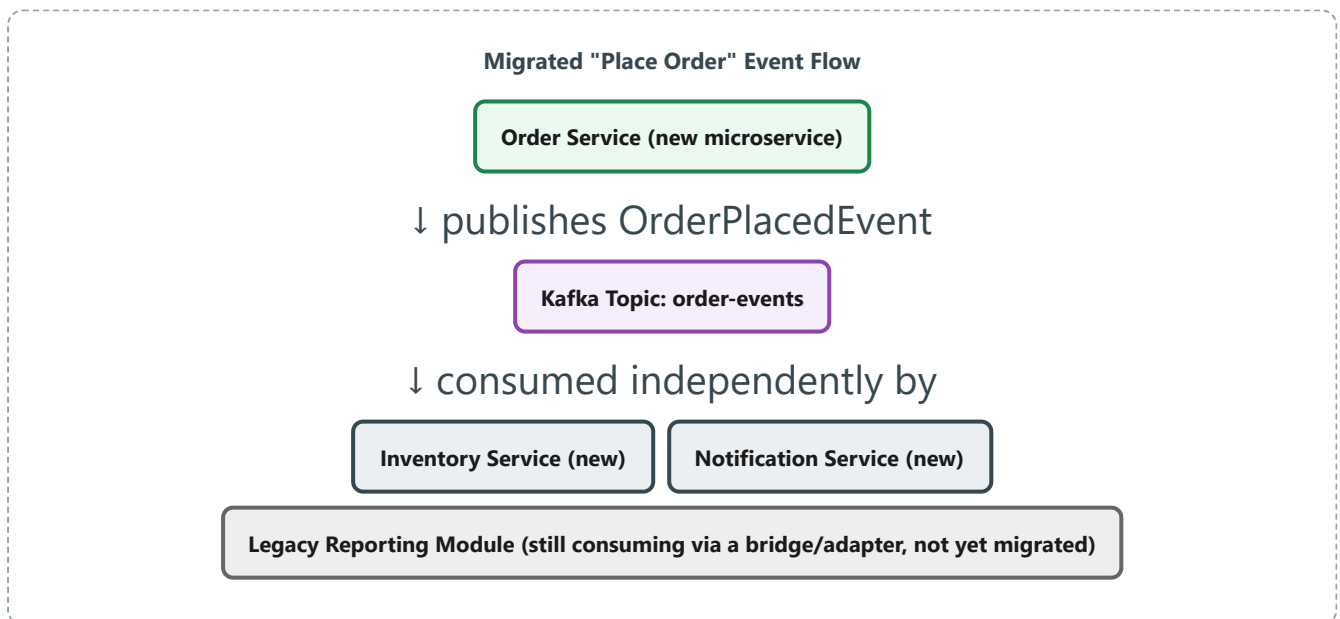
13.1 Why Kafka Becomes Necessary After Decomposition

In the monolith, "when an order is placed, update inventory and notify the customer" was just sequential method calls within one process. After decomposition (Section 10), these are separate services — Kafka provides the asynchronous, decoupled communication mechanism replacing those in-process calls (full pattern detail: Microservices guide, Section 24).

13.2 Real Kafka Adoption Rollout Strategy

1. **Introduce Kafka infrastructure early** (often during Phase 1-2), even before the first business event flows through it — validate the platform itself first.
2. **Start with non-critical events** — e.g., audit logging or notification triggers — where a temporary issue with the event pipeline has low business impact.
3. **Gradually adopt Kafka for core business events** (order placed, payment completed) only once the team has real operational confidence in the platform.
4. **Maintain synchronous REST fallbacks initially** for the most critical flows during early adoption, only fully committing to async-only once proven stable.

13.3 Real Migrated Event Flow



Real scenario: "During our migration's middle phase, our new Order Service published events to Kafka, but our NOT-YET-migrated legacy reporting module (still running in the monolith) needed that same data — we built a small adapter service that consumed the Kafka events and wrote them into the legacy database's expected format, letting the old reporting module keep working unmodified until it was its own turn to be migrated."

14. Phase 7: Frontend Migration — JSP to React

14.1 The Real Challenge — Two Frontends Coexisting

Since backend capabilities migrate incrementally (Strangler Fig, Section 8), the frontend often must ALSO support a transition period where some pages are still legacy JSP and others are the new React SPA — real teams handle this via a routing layer that directs users to the correct frontend based on which feature they're accessing.

14.2 Real Frontend Migration Approaches

Approach	How It Works	Real Trade-Off
Page-by-Page Replacement	Each JSP page is replaced by an equivalent React page/route as its backend capability migrates	Simple mental model, but users may experience jarring navigation between old-look and new-look pages during the transition
Micro-Frontend Approach	React components are embedded WITHIN existing JSP pages incrementally, sharing the same overall page shell	Smoother visual transition for users, but adds real integration complexity (shared styling, routing coordination)
Full Frontend Cutover First	Build the complete new React SPA against a mix of new and (temporarily) legacy-proxied APIs, replacing the ENTIRE frontend at once	Cleanest user experience, but requires the React app to work correctly against not-yet-fully-migrated backend capabilities

Most real enterprise migrations use **Page-by-Page Replacement** combined with the same Strangler Fig routing layer used for the backend (Section 8.3) — as each backend capability migrates, its corresponding JSP page is simultaneously replaced with a new React page calling the new REST API.

14.3 Real React Structure for This Migration

```
customer-portal/
├── src/
│   ├── pages/
│   │   ├── ProfilePage.tsx    # NEW — replaces legacy customerProfile.jsp
│   │   ├── OrderHistoryPage.tsx # NEW — replaces legacy orderHistory.jsp
│   │   └── LegacyRedirect.tsx  # redirects not-yet-migrated routes back to the legacy JSP app
│   └── api/
│       └── generated-client/  # typed client generated from the new services' OpenAPI specs
```

14.4 Real Data Fetching Differences (JSP vs React)

Aspect	Legacy JSP	New React
Rendering	Server-side rendered, full page reload per navigation	Client-side rendered, API calls for data, no full page reload
State management	Session-based (server-side HttpSession)	Client-side state (React Context/Redux), stateless JWT-based auth (Web/Network guide, Section 7)

Form handling	Struts ActionForm + server-side validation	React form libraries + client-side validation, backed by Spring Boot's Bean Validation (API Development guide, Section 15)
---------------	--	--

15. Phase 8: Containerization (Docker)

15.1 What Gets Containerized

Every new Spring Boot microservice and the new React frontend get their own Dockerfile (full detail: Docker & Kubernetes guide, Sections 8-9) — the legacy Struts monolith is typically NOT containerized in most real migrations (not worth the investment for a system being actively decommissioned), and instead continues running on its existing Tomcat infrastructure until fully retired.

15.2 Real Dockerfile for a Migrated Service

```
# Exactly the same multi-stage pattern from the Docker & Kubernetes guide, Section 8.2
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests -B

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/customer-service-*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

15.3 Real Decision — Should the Legacy Monolith Also Be Containerized?

Option	When It Makes Sense
Containerize the legacy monolith too	If the migration will take YEARS and the organization wants consistent deployment tooling across old and new systems during the long transition
Leave the legacy monolith on its existing infrastructure	If the migration timeline is shorter, or the effort to containerize a system being actively decommissioned isn't worth the investment

Real scenario: "Our organization chose to containerize even the legacy Struts monolith (a straightforward Tomcat + WAR Docker image, no code changes) purely to standardize our deployment pipeline and get it running on the SAME Kubernetes cluster as our new services — this let us use the SAME routing layer (Ingress) for the Strangler Fig pattern, rather than maintaining two completely separate infrastructure stacks during the transition."

16. Phase 9: Kubernetes Deployment (AKS)

16.1 Real AKS Cluster Setup for This Migration

A single AKS cluster typically hosts BOTH the legacy monolith (if containerized, Section 15.3) and all new microservices, using separate namespaces for isolation — exactly the namespace-per-environment/team pattern from the Docker & Kubernetes guide, Section 19.3, applied here as namespace-per-migration-phase.

16.2 Real Ingress Configuration Implementing the Strangler Fig Routing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: { name: migration-routing-ingress, namespace: prod }
spec:
  rules:
  - host: app.company.com
    http:
      paths:
      - path: /profile
        pathType: Prefix
        backend: { service: { name: customer-service, port: { number: 80 } } } # NEW microservice
      - path: /orders
        pathType: Prefix
        backend: { service: { name: order-service, port: { number: 80 } } } # NEW microservice
      - path: /
        pathType: Prefix
        backend: { service: { name: legacy-monolith, port: { number: 8080 } } } # everything else still legacy
```

This single Ingress resource IS the Strangler Fig routing layer in Kubernetes-native form — migrating a new capability is literally adding one new path rule pointing to the new service, with zero changes needed to the legacy monolith itself.

16.3 Real Resource Planning During Migration

Early in the migration, new microservices handle only a small fraction of total traffic — real teams start with conservative resource requests/limits and modest HPA min/max ranges, scaling these up progressively as each new service takes on more real traffic through subsequent migration waves (Docker & Kubernetes guide, Section 20.1).

17. Phase 10: Azure Services Selection

17.1 Real Azure Service Mapping for This Migration

Need	Azure Service Chosen	Why
Container orchestration	AKS	Runs both legacy (if containerized) and new microservices consistently
Container registry	ACR	Stores all Docker images built during the migration
Frontend hosting	Azure Static Web Apps (for the new React app)	Global CDN-backed hosting, matches the target architecture (Section 4)
New service databases	Azure SQL Database / PostgreSQL Flexible Server	Managed relational databases for each newly split service (Section 11)
Event streaming	Event Hub (Kafka-compatible endpoint)	Supports Kafka adoption (Section 13) without operating Kafka brokers directly
Secrets/config	Key Vault	Centralizes credentials for BOTH legacy and new systems during the transition period
Identity	Entra ID	Unified identity for new services; may need to bridge with legacy authentication initially

17.2 Real Legacy Database Hosting Decision

The legacy shared database is often migrated to Azure (e.g., "lift and shift" via Rehost, per the Traditional/Hybrid/Cloud guide's 6 R's, Section 20) EARLY in the program, even before application code migrates — this gets the organization onto Azure's infrastructure/networking foundation immediately, and the CDC-based database splitting (Section 11.3) can then happen entirely within Azure rather than across an on-premises-to-cloud boundary.

18. Phase 11: CI/CD Setup

18.1 Real Pipeline Strategy During Migration

Each new microservice gets its OWN independent Azure DevOps pipeline (exactly the standardized template pattern from the CI/CD guide, Section 20 and the AWS/Azure guide, Section 32.3) — while the legacy monolith typically keeps its EXISTING deployment process initially, since investing in modernizing a system being actively decommissioned is rarely worth the effort, UNLESS the migration timeline is very long.

18.2 Real Pipeline Stages for a Newly Migrated Service

```
trigger: { branches: { include: [ main ] }, paths: { include: [ customer-service/* ] } }
stages:
- stage: Build
  jobs: [ { job: BuildTest, steps: [ Maven build, unit tests, SonarQube quality gate ] } ]
- stage: Docker
  jobs: [ { job: DockerBuild, steps: [ Docker build, Trivy security scan, push to ACR ] } ]
- stage: DeployDev
  jobs: [ { deployment: Deploy, environment: 'Development', steps: [ KubernetesManifest deploy ] } ]
- stage: DeployProd
  jobs: [ { deployment: Deploy, environment: 'Production', steps: [ KubernetesManifest deploy ] } ] # approval-
gated
```

This is EXACTLY the real Azure DevOps pipeline pattern already covered in complete depth in the CI/CD guide (Section 17.5) — nothing migration-specific changes about the pipeline mechanics themselves, only that this pipeline now exists PER newly migrated service, growing in number as the migration progresses through its waves.

19. Phase 12: Observability & Monitoring Setup

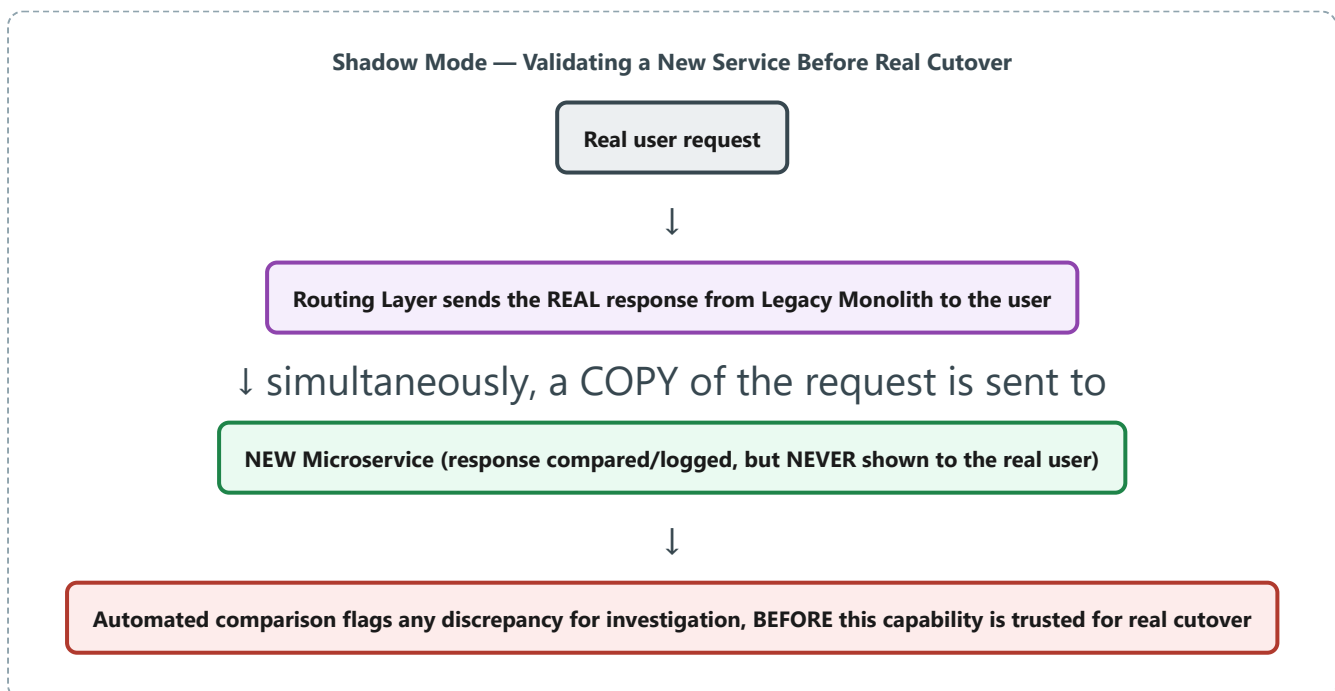
19.1 Why Observability Must Be Established EARLY, Not Late

A very common real migration mistake is deferring observability setup until "later" — without it, comparing the NEW system's behavior/performance against the legacy system's during the critical parallel-running period (Strangler Fig, Section 8) becomes guesswork instead of data-driven confidence.

19.2 Real Observability Needs Specific to a Migration

Need	Real Approach
Compare new microservice performance against the legacy baseline	Track the SAME business-level metrics (e.g., "order creation latency") on both systems side by side during the transition
Trace a request that crosses BOTH legacy and new systems	Propagate a correlation ID even through the legacy monolith (often requiring a small code addition to the otherwise-frozen legacy codebase) so traces remain connected end to end
Detect a migrated capability behaving incorrectly compared to the legacy version	Run the new service in "shadow mode" initially — receiving real traffic copies but not serving the actual response — comparing its output against the legacy system's real response before fully cutting over

19.3 Real Shadow Mode Testing Pattern



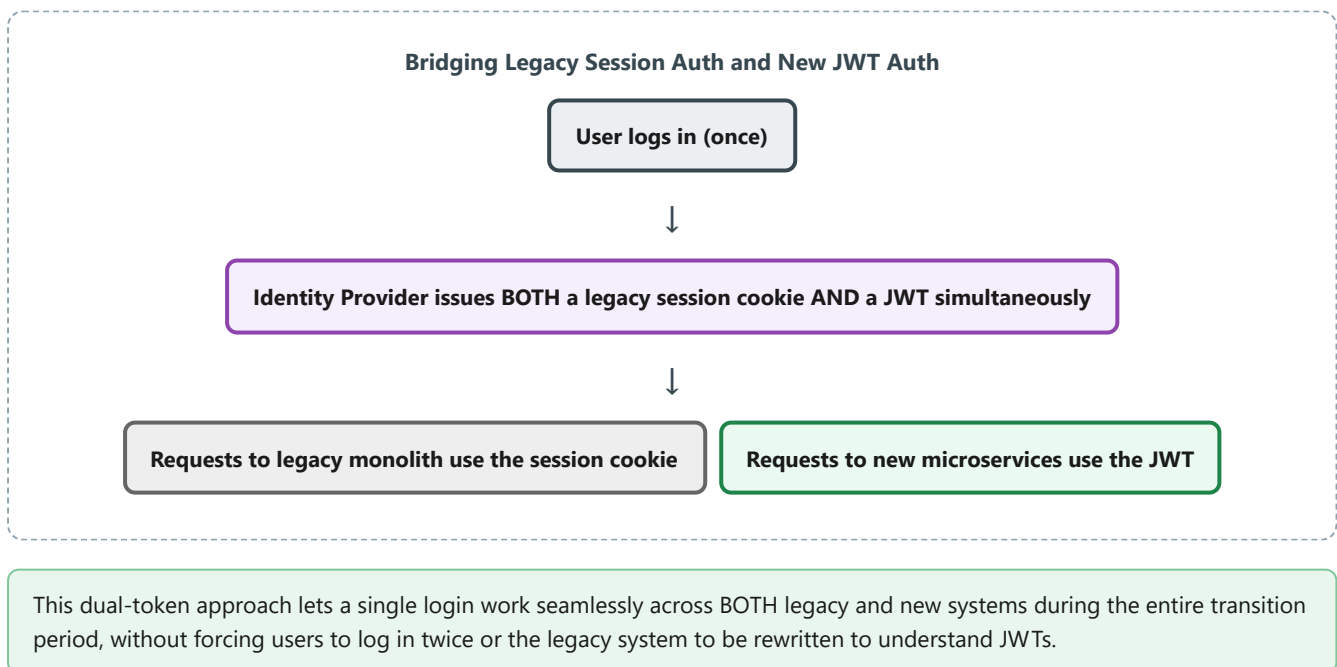
Real scenario: "Before cutting over our Order History feature, we ran the new Order Service in shadow mode for two full weeks — comparing its responses against the legacy JSP page's output for every real production request. This caught a subtle date-formatting discrepancy and a pagination edge-case bug BEFORE any real customer ever saw the new service's actual response, turning what could have been a customer-facing incident into a quiet pre-launch fix."

20. Phase 13: Security Implementation

20.1 Real Security Challenges Unique to a Migration

Challenge	Real Solution
Legacy system uses server-side sessions; new system uses stateless JWTs	Build a bridging authentication layer during the transition — often the API Gateway/APIIM validates the SAME identity provider's token for both old and new systems, or a shim translates between session cookies and JWTs
Legacy Struts framework has known historical vulnerabilities	Prioritize migrating (or at minimum, patching/isolating) internet-facing legacy components with known CVEs, even ahead of their "natural" migration wave order if the risk is severe
New microservices need Managed Identity/RBAC; legacy uses hardcoded/shared credentials	New services adopt Managed Identity from day one (AWS/Azure guide, Section 9.5); legacy credentials are rotated and secured in Key Vault even if not fully modernized
Expanded attack surface — many new microservices instead of one monolith	Enforce API Gateway-level authentication/rate-limiting centrally (API Development guide, Section 21) so every new service inherits consistent security posture automatically

20.2 Real Authentication Bridging Pattern



21. Phase 14: Production Cutover Strategy

21.1 The Real Cutover Process Per Capability

- 1. **Shadow mode validation** (Section 19.3) — run the new service against real traffic copies, comparing output, for a defined observation period.
- 2. **Canary cutover** — route a small percentage (e.g., 5%) of REAL traffic to the new service, closely monitoring error rates/latency (Docker & Kubernetes guide, Section 16.5).
- 3. **Gradual ramp-up** — increase to 25% → 50% → 100% over hours/days, only proceeding if metrics remain healthy at each stage.
- 4. **Full cutover** — 100% of traffic for this capability now goes to the new microservice; the legacy code path remains dormant but available for a defined rollback window (Section 22).
- 5. **Legacy code retirement** — after a sufficient stable period (often 1-3 months), the legacy code for this specific capability is finally removed.

21.2 Real Cutover Decision Criteria

Metric	Real Threshold for Proceeding
Error rate	New service's error rate must be equal to or better than the legacy baseline
Latency (p95/p99)	New service's latency must be within an agreed acceptable range of the legacy baseline
Data consistency	Zero discrepancies found during the shadow mode comparison period for this capability
Business sign-off	Business stakeholders formally approve the cutover for customer-facing, high-value capabilities

22. Rollback Strategies

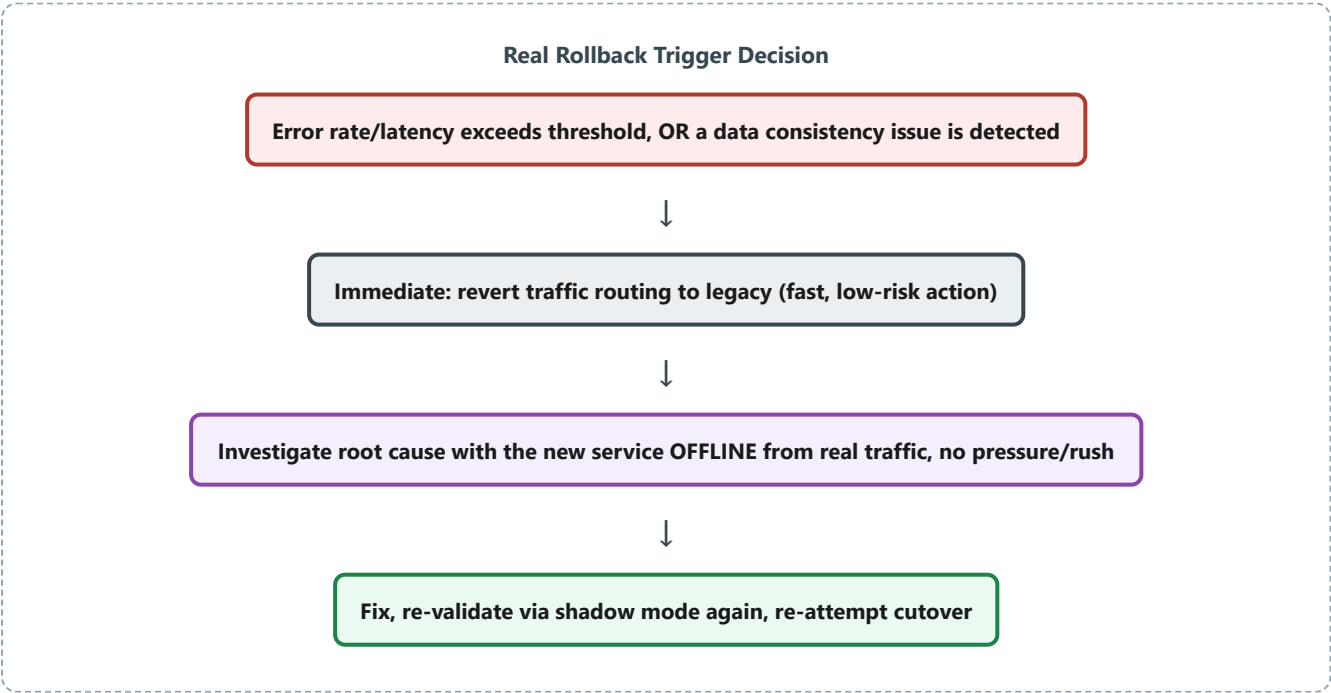
22.1 Why Rollback Planning Is Non-Negotiable

Every single cutover step (Section 21) MUST have a pre-planned, tested rollback path BEFORE it's executed in production — "we'll figure out rollback if something goes wrong" is exactly the kind of thinking that turns a minor issue into a major incident.

22.2 Real Rollback Mechanisms Per Layer

Layer	Real Rollback Mechanism
Traffic routing (Strangler Fig)	Revert the Ingress/APIM routing rule back to the legacy monolith — typically a near-instant configuration change (Section 16.2)
New microservice deployment	Kubernetes rollout undo to the previous stable version (Docker & Kubernetes guide, Section 21.4)
Database (if already cut over)	If CDC-based sync (Section 11.3) is still active/reversible, fall back to the legacy database as source of truth; if not, this is why data reconciliation validation (Section 11.4) must happen BEFORE fully retiring the legacy database
Frontend	Revert the frontend routing to serve the legacy JSP page again for the affected feature

22.3 Real Rollback Decision Framework



Real scenario: "During our Payment Service cutover, a canary at 5% traffic revealed a subtle currency rounding discrepancy compared to the legacy system — we reverted the APIM routing rule back to 100% legacy within two minutes of the alert"

firing, with zero customer impact, then spent the next week fixing and re-validating the rounding logic in shadow mode before successfully re-attempting the cutover."

23. System Design — Where to Apply Each Pattern (Decision Framework)

23.1 A Practical Decision Framework for This Migration

System Design Question	Decision Framework
Should this capability be its own microservice?	Does it have its own data, its own business rules, and a genuine reason to scale/deploy independently? If not, keep it grouped with a related capability rather than over-decomposing.
Should this operation be synchronous (REST) or asynchronous (Kafka)?	Does the caller need an IMMEDIATE answer to proceed (e.g., "is this account valid?")? Use REST. Is it a side effect that can tolerate a short delay (e.g., "send a confirmation email")? Use Kafka.
Does this cross-service operation need a Saga?	Does it span multiple services' databases and require all-or-nothing correctness? Use Saga with compensating transactions. Is it read-only or single-service? No Saga needed.
Should we use CDC or dual-write for database migration?	Is minimizing downtime critical and is engineering capacity available for CDC's added complexity? Use CDC. Is a short, well-communicated maintenance window acceptable? A simpler Big Bang split may suffice.
Should this capability go through shadow mode before cutover?	Is it a high-traffic, high-business-value, or complex-logic capability? Always shadow-test. Is it a low-risk, simple, low-traffic feature? Shadow mode may be a lower priority, faster direct canary may suffice.
Pessimistic or optimistic locking for a migrated inventory/balance operation?	Is this a high-contention, high-value operation (large fund transfer)? Consider pessimistic locking. Is contention genuinely rare? Optimistic locking with retry is usually more scalable (JDBC/JPA guide, Section 17).

23.2 Real System Design Interview Framing

When asked in an interview to "design the migration of a legacy monolith," structure your answer around: (1) Assessment first, never skip it, (2) Strangler Fig for incremental, low-risk cutover, (3) Database per Service + Saga for the resulting distributed data challenge, (4) Shadow mode + canary + clear rollback triggers for EVERY cutover step, and (5) observability established EARLY, not as an afterthought — this demonstrates real, practiced migration judgment, not just knowledge of the target technology stack.

24. Real-Time Challenges During Migration

Challenge	Why It's Genuinely Hard	Real Mitigation
Keeping the legacy system stable while migrating	The legacy codebase still needs bug fixes and even occasional new features during the (long) migration period, competing for the same developers' time	Dedicate a small, separate "legacy maintenance" team distinct from the migration team, or apply strict feature-freeze policies for lower-priority legacy areas
Data consistency during the parallel-running period	Two representations of the same data (legacy shared DB, new per-service DBs) must stay consistent for weeks/months	CDC-based synchronization (Section 11.3) with automated reconciliation checks running continuously, not just once
Team morale and fatigue over a multi-year program	Migration work can feel like "unglamorous plumbing" compared to building new features, causing attrition risk	Celebrate and visibly communicate each successful migration wave; rotate developers between migration work and new feature work where possible
Hidden business logic buried in JSP scriptlets	Years of undocumented edge-case handling accumulated directly in view-layer code, easy to miss during decomposition	Extensive automated regression testing (Section 7.2) comparing old vs new behavior across a wide range of real historical inputs
Scope creep — "since we're rewriting it, let's also redesign the business process"	Mixing a technical migration with simultaneous business process changes multiplies risk and makes shadow-mode comparison (Section 19.3) meaningless (comparing different behaviors, not verifying equivalence)	Enforce a strict rule: migrate behavior AS-IS first, then separately propose/implement business process improvements afterward, as their own initiative
Underestimating the database migration effort	Splitting a decade-old, organically-grown shared schema is almost always harder and slower than initially estimated	Allocate significant, realistic time and senior DBA involvement specifically for Section 11's database splitting work — a very common source of real schedule overruns

25. Advantages & Disadvantages of the Migration

25.1 Real Advantages

- Independent deployability — teams ship features without coordinating a monolith-wide release.
- Independent scalability — scale only the specific services under real load (e.g., Inventory Service during a sale).
- Modern, responsive React UI improves customer experience and developer productivity.
- Improved security posture — retiring an outdated framework with known historical vulnerabilities.
- Better observability — full distributed tracing/metrics versus legacy basic log files.
- Easier hiring — modern tech stack (Spring Boot, React, Kubernetes) is far easier to recruit developers for than legacy Struts.
- Cloud elasticity — auto-scaling and pay-as-you-go infrastructure versus fixed on-premises capacity.

25.2 Real Disadvantages / Honest Costs

- Very high upfront investment — significant time, budget, and organizational focus over 1-3+ years.
- Real operational risk during the transition — running two systems in parallel is genuinely more complex than running one.
- Distributed systems complexity introduced where none existed before (network latency, partial failures, eventual consistency — Microservices guide, Section 6.2).
- Requires significant new skill investment across the whole organization (Kubernetes, Kafka, cloud-native patterns).
- Risk of scope creep and schedule overrun if not tightly governed (Section 24).
- Short-term productivity dip — teams learning new patterns/tools are initially slower than they were in the familiar (if painful) legacy system.

25.3 The Honest Bottom Line

Interview one-liner: "A migration like this trades short-term pain (cost, risk, complexity) for long-term agility (faster feature delivery, independent scaling, better talent retention) — it's the right call when the legacy system's ongoing cost/risk clearly exceeds the migration's cost/risk, and a poor call when done simply because microservices are 'trendy' without that honest cost-benefit analysis having actually been done."

26. Common Production Issues After Migration & Solutions

Issue	Cause	Real Solution
A migrated feature is slower than the legacy version	Added network hops (API Gateway → microservice → database) versus the monolith's in-process calls; possible N+1 query issues in the newly written JPA code	Profile via distributed tracing (Observability guide) to find the actual slow hop; check for N+1 problems (JDBC/JPA guide, Section 13); add caching for read-heavy data
Data discrepancy discovered between legacy and new systems post-cutover	A CDC/dual-write synchronization gap not caught during shadow mode validation	Immediately roll back this capability's traffic routing (Section 22), investigate and fix the sync gap, re-validate via a longer shadow mode period before re-attempting cutover
Users experience session/login issues during the transition	The legacy session vs new JWT authentication bridge (Section 20.2) has an edge case not handled correctly	Verify the identity provider issues BOTH token types consistently; add explicit fallback/re-authentication handling for edge cases (e.g., session expiry timing mismatches)
A migrated microservice cascades failures to unrelated features	Missing circuit breakers/timeouts on a newly introduced inter-service call not present in the original monolith's in-process design	Apply the full resilience pattern set (Microservices guide, Section 40) — this is a NEW failure mode introduced specifically by decomposition, and must be explicitly designed for
Kafka consumer lag grows unexpectedly after a new capability's cutover	The new consumer's processing logic is slower than the actual event production rate at real traffic volume	Scale consumer instances, increase topic partitions for parallelism, or optimize the consumer's processing logic (SQL/Spring Batch guide parallels)
Business team reports "the new system behaves differently" for an edge case	An undocumented legacy business rule (buried in old JSP scriptlets) wasn't captured during Assessment/decomposition	This is exactly why shadow mode (Section 19.3) and extensive regression testing exist — investigate, restore correct behavior, add a regression test for this specific case going forward

27. Real-World Migration Scenarios

Scenario 1 — The Successful Phased Migration: "A large retail bank migrated its Struts-based online banking portal over 30 months, using Strangler Fig with 12 distinct migration waves. Each wave included assessment, shadow mode validation, canary cutover, and a mandatory 4-week stability observation period before the next wave began. By the end, all core banking features (Accounts, Cards, Loans — the very domains covered throughout this documentation set) ran as independent Spring Boot microservices on AKS, with the original Struts monolith fully decommissioned and zero customer-facing outages attributed to the migration itself."

Scenario 2 — A Cautionary Tale of Rushed Migration: "A different organization attempted to migrate a similarly-sized system in just 6 months, skipping shadow mode testing to 'save time' and cutting over multiple high-risk capabilities simultaneously. A subtle data consistency bug in the payment flow (missed without shadow mode comparison) caused a real production incident affecting thousands of customers, requiring an emergency full rollback to the legacy system and a 4-month program pause to rebuild proper validation processes before re-attempting — a costly lesson in why the phases and safeguards in this guide exist."

Scenario 3 — Steel Manufacturing ERP Modernization (Cross-Reference): "A steel manufacturer (see the Enterprise Domain Handbook, Section 3) modernized its legacy Struts-based production planning system using this exact same playbook — Assessment identified the Production Planning and Quality Management modules as the highest-value, lowest-risk starting point (Wave 1), while the deeply legacy-integrated MRP/ERP financial modules were deliberately migrated last, given their tight coupling to the existing SAP system and lower urgency."

28. Interview Questions & Solutions

Q1. How would you approach migrating a legacy Struts/JSP monolith to microservices, end to end?

Start with a thorough Assessment phase to build a domain map, form a cross-functional team (business, architecture, dev, QA, DBA, DevOps), apply the Strangler Fig pattern for incremental cutover starting with low-risk capabilities, use Database per Service with CDC-based splitting and Saga for distributed transactions, validate every capability via shadow mode before a gradual canary cutover, and maintain clear rollback triggers at every step — establishing observability and CI/CD early, not as an afterthought.

Q2. Why is a "big bang" migration approach considered risky, and what's the alternative?

A big bang cutover replaces the entire system at once, with no way to isolate or safely roll back a single problematic capability if something goes wrong — the Strangler Fig pattern instead migrates one capability at a time, each independently validated and reversible, keeping blast radius small for any single issue.

Q3. How do you handle a distributed transaction that used to be a single local transaction in the monolith?

Replace it with the Saga pattern — break the operation into a sequence of local transactions across the now-separate services, each with a defined compensating action to undo it if a later step fails, trading strict atomicity for eventual consistency with guaranteed rollback logic.

Q4. What is shadow mode testing, and why is it valuable during a migration specifically?

Running the new service against a copy of real production traffic without actually serving its response to users, comparing its output against the legacy system's real response — this catches subtle behavioral discrepancies (formatting differences, edge-case bugs) before real customers are ever exposed to the new system, which is especially valuable when migrating undocumented legacy business logic.

Q5. How would you migrate a shared database used by a monolith into separate databases per microservice, with minimal downtime?

Use Change Data Capture (CDC, e.g., Debezium) to continuously replicate changes from the legacy database into the new per-service databases in near-real-time while the legacy system keeps running, run automated reconciliation checks to verify consistency, and only cut a service over to read/write its own new database once fully validated.

Q6. What role does each team (Business, Architecture, Dev, QA, DBA, DevOps) play in a migration like this?

Business prioritizes and approves cutover decisions; Architecture designs the target state and service boundaries; Dev implements the new services; QA builds regression coverage validating behavioral equivalence; DBA plans and executes the database splitting strategy; DevOps builds the CI/CD, Kubernetes, and observability foundation every migrated service depends on.

Q7. How do you handle authentication when the legacy system uses server-side sessions and the new system uses JWT?

Have a single identity provider issue BOTH a legacy session cookie and a JWT at login time, so requests to the still-running legacy monolith use the session cookie while requests to new microservices use the JWT — allowing one login to work seamlessly across both systems throughout the transition.

Q8. What's your rollback plan if a newly migrated capability starts failing in production?

Immediately revert the Strangler Fig routing rule (Ingress/API Gateway) back to the legacy monolith — a fast, low-risk action — taking the new service out of the real traffic path entirely, then investigate and fix the root cause offline before re-validating via shadow mode and re-attempting a canary cutover.

Q9. What are the real disadvantages of this kind of migration that you'd communicate honestly to stakeholders?

Very high upfront cost and multi-year timeline, real operational complexity from running two systems in parallel, genuinely new distributed-systems failure modes (network partial failures, eventual consistency) that didn't exist in the monolith, and a short-term productivity dip while teams learn new tools and patterns — all weighed against the long-term agility, scalability, and talent-retention benefits.

Q10. Describe a real production issue that could occur specifically because of this migration, and how you'd resolve it.

A migrated feature becomes noticeably slower than its legacy equivalent due to added network hops between the new microservices, or an N+1 query issue in newly written JPA code — resolved by using distributed tracing to pinpoint exactly which hop is slow, then applying the appropriate fix (JOIN FETCH/Entity Graph for N+1, caching for read-heavy data, or circuit breakers/timeouts if the issue is a downstream dependency).

29. Final Summary & Cheat Sheet

The legacy state: a Struts/JSP monolith with mixed business logic, raw JDBC, one shared database, manual deployments, and mounting technical/security debt.

The target state: React + Spring Boot microservices, database per service, Kafka-driven event communication, containerized on AKS, deployed via Azure DevOps CI/CD, secured with Key Vault/Entra ID, and observed via Azure Monitor/Application Insights.

The core migration patterns: Strangler Fig (incremental, reversible capability-by-capability cutover), Database per Service with CDC-based splitting (solving the shared-database problem), and Saga (replacing the monolith's local transactions across newly independent services).

The non-negotiable safeguards: thorough upfront Assessment, cross-functional team governance, shadow mode validation before every cutover, canary-based gradual traffic ramp-up, and a clear, tested rollback plan at every single step.

The honest trade-off: significant upfront cost, risk, and complexity in exchange for long-term agility, scalability, security, and talent-retention benefits — the right call when a genuine cost-benefit analysis supports it, not simply because modern technology is more exciting to work with.

29.1 Quick Cheat Sheet

Term	One-Line Meaning
Strangler Fig Pattern	Incrementally route traffic away from the monolith, one capability at a time
Domain Map	The output of Assessment — business capabilities mapped to future service boundaries
CDC (Change Data Capture)	Real-time database replication enabling near-zero-downtime data splitting
Saga Pattern	Sequence of local transactions + compensations replacing a lost distributed transaction
Shadow Mode	Running a new service against real traffic copies without serving its response, to validate behavior
Canary Cutover	Gradually ramping real traffic percentage to a newly migrated service
Migration Wave	A batch of capabilities migrated together, ordered from lowest to highest risk
Dual-Write Risk	The danger of legacy and new databases drifting out of sync during transition
Authentication Bridge	Issuing both a legacy session cookie and a JWT so one login works across both systems
Rollback Trigger	A predefined metric threshold that automatically/manually reverts a cutover

Use this guide alongside the Microservices guide, the Docker & Kubernetes guide, the CI/CD guide, the AWS/Azure Cloud Architecture guide, the Observability & Monitoring guide, and the Enterprise Domain Handbook — together they cover the complete real-world journey of taking a legacy Java enterprise system and modernizing it into a production-grade, cloud-native microservices architecture.